

A NO-FLUFF HANDBOOK

Patty's Guide to GitHub

From your first commit to your first pull request

by Pathin

Contents

Before you start 3

THE BASICS

1 The mental model 4

2 One-time setup 5

3 Getting a repo onto your machine 6

4 The daily loop 7

WORKING TOGETHER

5 Branches 8

6 Working together on GitHub 9

7 Merge conflicts 11

FIXING THINGS

8 Undo cookbook 12

9 Reading history 13

GOOD PRACTICE

10 Files every repo should have 14

11 Habits that save you 15

LOOK IT UP

12 Cheat sheet 16

13 Glossary 18

TRY IT

14 Practice session (do it together) 19

READ THIS FIRST

Before you start

This book is short on theory and long on commands you'll actually type. Chapters 1 to 4 are enough to start saving and sharing work; everything after builds on them.

Part	Chapters	Read it
The basics	1. The mental model · 2. One-time setup · 3. Getting a repo · 4. The daily loop	First, in order
Working together	5. Branches · 6. Working together on GitHub · 7. Merge conflicts	Once the basics feel easy
Fixing things	8. Undo cookbook · 9. Reading history	When something goes wrong
Good practice	10. Files every repo should have · 11. Habits that save you	Before your first real project
Look it up	12. Cheat sheet · 13. Glossary	Anytime
Try it	14. Practice session	Together, after chapter 7

Every command works on Mac, Windows (in Git Bash) and Linux. Anything after a `#` in a command is a note to you; you don't type it.

CHAPTER 1

The mental model

Git saves snapshots of a project on your computer; GitHub stores a copy online so people can share it and work on it together. Git works fine offline. GitHub is the meeting point.

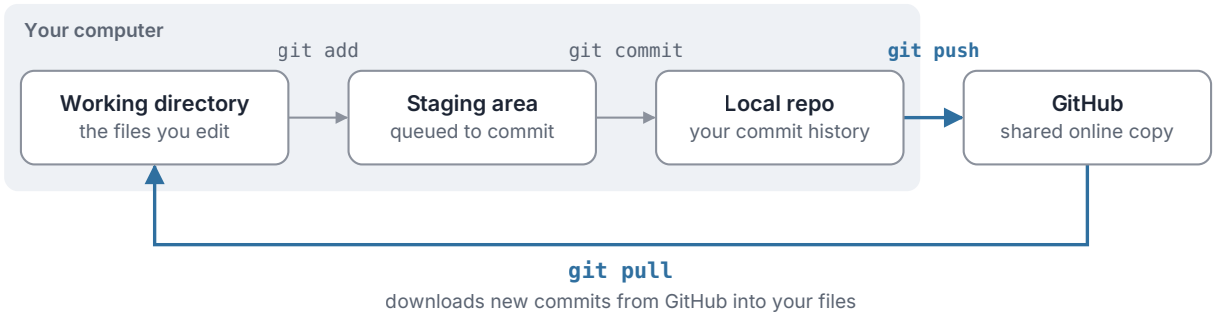
Term	What it actually is
Repository (repo)	A project folder Git tracks, plus its full history (kept in a hidden `.git` folder)
Commit	A saved snapshot of the project, with a message and an ID like `a1b2c3d`
Branch	A named line of commits that lets you try things without touching `main`
`main`	The default branch: the version that should always work
Remote	A copy of the repo somewhere else, usually GitHub, named `origin` by default
Push / Pull	Push uploads your commits to the remote; pull downloads new ones from it

Every change moves through three places on your machine before it reaches GitHub:

1. **Working directory**: the files you see and edit.
2. **Staging area**: the changes you picked for the next commit (`git add`).
3. **Local repo**: the committed history on your computer (`git commit`).

Nothing reaches GitHub until you `git push`, and nothing from GitHub reaches you until you `git pull`. Most beginner confusion comes from forgetting one of those two.

Changes reach GitHub only when you push, and come back when you pull



Read it left to right: `git add` and `git commit` only change things on your computer. `git push` is the one step that sends anything to GitHub.

CHAPTER 2

One-time setup

Do this once per computer. It takes about 10 minutes.

1. **Make a GitHub account** at github.com.
2. **Install Git.**
   - Mac: run `git --version` in Terminal. If Git is missing, macOS offers to install it. Or use `brew install git`.
   - Windows: install Git for Windows from git-scm.com. Use the "Git Bash" app it adds.
   - Linux: `sudo apt install git` (Ubuntu/Debian).
3. **Tell Git who you are.** This name and email go on every commit. Use the same email as your GitHub account so commits link to your profile.
4. **Log in from your computer.** GitHub does not accept your account password for Git commands. Use the GitHub CLI (easiest) or an SSH key.
5. **Check it worked:** `git config --list` shows your settings.

Step 3: who you are

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

Step 4, option A: GitHub CLI

Install `gh` from cli.github.com, then run `gh auth login`. Pick GitHub.com, HTTPS, and log in with your browser.

Step 4, option B: SSH key

Create a key, copy the public half, and paste it into GitHub under Settings, then SSH and GPG keys, then New SSH key.

```
ssh-keygen -t ed25519 -C "you@example.com"   # press Enter through the prompts
cat ~/.ssh/id_ed25519.pub                     # copy this whole line into GitHub
ssh -T git@github.com                         # test: should greet you by username
```

> **Prefer buttons over a terminal?** GitHub Desktop (desktop.github.com) or the Source Control panel in VS Code do everything in this book with clicks. The concepts stay the same.

CHAPTER 3

Getting a repo onto your machine

Pick the starting point that matches your situation. Option A is the easiest for a first project.

A. Start on GitHub, then clone

1. On GitHub, click **+** (top right), then **New repository**.
2. Name it, tick **Add a README file**, click **Create repository**.
3. Click the green **Code** button and copy the URL.
4. Download it to your computer:

```
git clone https://github.com/you/my-project.git
cd my-project
```

B. You already have a project folder

Turn the folder into a repo and make the first commit:

```
cd my-project
git init
git add .
git commit -m "Initial commit"
```

Then create an **empty** repo on GitHub (no README, so the two histories don't clash) and connect it:

```
git remote add origin https://github.com/you/my-project.git
git push -u origin main
```

The `-u` links your local `main` to GitHub's, so later you can type just `git push` and `git pull`. With the GitHub CLI, one command does all of B: `gh repo create my-project --private --source=. --push`.

C. Someone else's repo

Your access	What to do
You're a collaborator (they added you)	Clone it directly, like option A
You're not	Click **Fork** on GitHub to get your own copy, then clone your fork

CHAPTER 4

The daily loop

This is 90% of Git. Run these five steps every time you work on the project.

```
git pull                            # 1. get the latest from GitHub
# ...edit your files...
git status                          # 2. see what changed
git add index.html                  # 3. stage a file (git add . stages everything)
git commit -m "Add contact form"    # 4. save a snapshot
git push                            # 5. upload to GitHub
```

`git status` is your dashboard. Run it whenever you're unsure, and read what it says:

`git status` says	It means	Do this
Untracked files	New files Git has never seen	`git add` them, or list them in `.gitignore`
Changes not staged for commit	Edited but not added	`git add`
Changes to be committed	Staged and ready	`git commit`
Your branch is ahead of 'origin/main'	Committed but not pushed	`git push`
Your branch is behind 'origin/main'	GitHub has newer commits	`git pull`
nothing to commit, working tree clean	Everything is saved	Nothing

To see the actual line changes before committing: `git diff` (not yet staged) and `git diff --staged` (staged).

Rules that prevent pain

- Pull before you start, push when you stop.
- Check `git status` before `git add .` so you don't commit junk.
- Commit small and often. A commit is a save point you can return to.

CHAPTER 5

Branches

A branch lets you build something without breaking `main`. Rule: `main` always works, and new work happens on a branch. Branches are free, so make one whenever you start something.

```
git branch                              # list branches (* = the one you're on)
git switch -c feature/navbar            # create a new branch and move to it
git switch main                         # move back to main
git push -u origin feature/navbar       # put the branch on GitHub

# finished? merge it into main:
git switch main
git merge feature/navbar

# clean up after merging:
git branch -d feature/navbar            # delete it locally
git push origin --delete feature/navbar # delete it on GitHub
```

- `git checkout -b name` is the older way to do `git switch -c name`. Tutorials use both; they do the same thing.
- Commit (or stash, see chapter 8) before switching branches. Otherwise Git may refuse, or carry your edits along.
- Name branches after what they do: `fix/login-typo`, `feature/dark-mode`.
- `git fetch` updates what Git knows about GitHub without touching your files. `git branch -a` then lists GitHub's branches too.
- `git pull` is just `git fetch` plus `git merge` in one step.

CHAPTER 6

Working together on GitHub

The pull request (PR) is the heart of GitHub: you propose changes from a branch, someone reviews them, then they get merged into `main`.

Setup	When	How
Collaborator	Friends or teammates on one shared project	Owner goes to the repo's **Settings**, then **Collaborators**, then **Add people**. Everyone clones the same repo and pushes branches to it.
Fork	Contributing to a repo you can't push to	Click **Fork**, clone your fork, push to your fork, then open a PR back to the original

The PR flow, start to finish

1. Make a branch and do the work: `git switch -c feature/x`, edit, commit.
2. Push it: `git push -u origin feature/x`.
3. On GitHub, click the **Compare & pull request** banner. Write a title and a short what/why. Click **Create pull request**. (Terminal option: `gh pr create`.)
4. The reviewer opens the **Files changed** tab, comments on lines, then chooses **Approve** or **Request changes**.
5. To fix feedback, commit and push to the same branch. The PR updates itself.
6. Click **Merge pull request**, then **Delete branch**.
7. Back on your computer: `git switch main`, `git pull`, `git branch -d feature/x`.

The three merge buttons

Option	What happens	Use it when
Create a merge commit	Keeps every commit and adds one merge commit	Default; safe choice
Squash and merge	All the PR's commits become one commit on `main`	The PR has lots of messy "fix typo" commits
Rebase and merge	Commits are replayed onto `main`, no merge commit	You want a perfectly straight history

Keep your branch up to date with main

Do this if `main` moved while you worked:

```
git switch main
git pull
git switch feature/x
git merge main
```

Fork users: sync with the original repo

Click **Sync fork** on your fork's GitHub page, or from the terminal:

```
git remote add upstream https://github.com/original-owner/repo.git   # once
git fetch upstream
git merge upstream/main
```

Other GitHub features worth knowing

Feature	What it does
Issues	A to-do and bug list attached to the repo
`Closes #12`	Written in a PR description, it auto-closes issue 12 when the PR merges
@mentions	`@username` in an issue or PR notifies that person
GitHub Pages	Hosts a static website straight from a repo, free
Actions	Runs automation, like tests on every push

CHAPTER 7

Merge conflicts

A conflict happens when two branches changed the same lines, so Git can't pick for you. It's normal, not a disaster: you decide what the final version is.

Git marks the clash right inside the file:

```
<<<<<<< HEAD
<h1>Welcome</h1>
=======
<h1>Hello there</h1>
>>>>>>> feature/header
```

The top half is your current branch. The bottom half is what's coming in.

Fix it

1. Run `git status`. Conflicted files show as "both modified".
2. Open each file and find the `<<<<<<<` markers.
3. Edit to the version you want: keep one side, the other, or a mix. Delete all three marker lines.
4. `git add <file>` for each fixed file.
5. `git commit` to finish the merge.
6. `git push`.

> **Panic button:** `git merge --abort` puts everything back to before the merge.

Easier ways: VS Code shows **Accept Current**, **Accept Incoming** and **Accept Both** buttons above each conflict. For simple cases, a PR on GitHub shows a **Resolve conflicts** button you can use in the browser.

To get fewer conflicts: pull often, keep branches short-lived, and tell each other before you both edit the same file.

CHAPTER 8

Undo cookbook

Find your situation and run the command. Rows marked **local only** rewrite history: use them only on commits you have not pushed yet.

Situation	Command	Safe after push?
Throw away edits to a file (not staged)	`git restore file.txt`	Yes
Unstage a file but keep the edits	`git restore --staged file.txt`	Yes
Typo in the last commit message	`git commit --amend -m "Better message"`	Local only
Forgot a file in the last commit	`git add file.txt` then `git commit --amend --no-edit`	Local only
Undo the last commit, keep the changes	`git reset --soft HEAD~1`	Local only
Undo the last commit and delete the changes	`git reset --hard HEAD~1` (destroys that work)	Local only
Undo a commit that's already on GitHub	`git revert <hash>` (adds a new commit that reverses it)	Yes
Park unfinished work to switch branches	`git stash`, later `git stash pop`	Yes
Committed a file that should be ignored	Add it to `.gitignore`, then `git rm --cached file` and commit	Yes
Lost a commit after a reset	`git reflog`, find the hash, then `git switch -c rescue <hash>`	Yes
Pushed a password or API key	Revoke and replace the key right away. Deleting it from history is not enough: assume it's leaked.	Act now

Committed to main but meant to use a branch (not pushed yet)

```
git switch -c my-feature          # new branch keeps your commits
git switch main
git reset --hard origin/main      # main goes back to match GitHub
```

`HEAD` means "the commit you're on now"; `HEAD~1` is the one before it. Get a commit's hash from `git log --oneline`.

> **The golden rule:** never rewrite commits other people already have (amend, reset, rebase, or `git push --force` on a shared branch). Use `git revert` instead. If you must force-push your own branch, use `git push --force-with-lease`, which refuses if someone else pushed in the meantime.

CHAPTER 9

Reading history

These answer "what changed, when, and who did it".

```
git log --oneline                # one line per commit, newest first
git log --oneline --graph --all  # same, with branches drawn as lines
git log -p file.txt              # every change ever made to one file
git show a1b2c3d                 # exactly what one commit changed
git diff                         # your edits not yet staged
git diff --staged                # your staged edits
git diff main..feature/x         # what feature/x has that main doesn't
git blame file.txt               # who last changed each line
```

Press `q` to exit the scrolling view that `git log` and `git diff` open.

On GitHub: the **Commits** link on the repo page shows the log. Open any file and click **History** or **Blame** for the same info in the browser.

CHAPTER 10

Files every repo should have

Three files make a repo usable by other people.

File	What it's for
`README.md`	The repo's front page on GitHub: what it is and how to run it. Written in Markdown.
`.gitignore`	Files Git should never track: secrets, dependencies, build output, OS junk
`LICENSE`	Says who may use the code. No license means nobody else legally can. On GitHub: **Add file**, name it `LICENSE`, then pick a template.

A starter .gitignore

One pattern per line; `#` is a comment.

```
# secrets
.env

# dependencies
node_modules/

# build output
dist/

# OS junk
.DS_Store
Thumbs.db
```

`.gitignore` only affects files Git isn't tracking yet. If a file was already committed, run `git rm --cached file` and commit. GitHub's `github/gitignore` repo has ready-made versions for most languages.

Markdown basics for the README

You type	You get
`# Title`	Big heading (`##` for smaller)
`**bold**`	**bold**
`- item`	Bullet point
`` `code` ``	Inline `code`
`[text](https://example.com)`	A link

CHAPTER 11

Habits that save you

A commit message says what the commit does, written as a command, in about 50 characters or less.

Bad	Good
stuff	Add contact form to homepage
fixed things	Fix broken link in navbar
asdfgh	Remove unused images
changes	Update README with setup steps

Need more detail? Run `git commit` with no `-m`. An editor opens: first line is the summary, then a blank line, then why you made the change. Stuck in Vim? Type your message, press Esc, then type `:wq` and press Enter.

The habits

- Pull before you start; push before you stop.
- One commit = one logical change. Easier to review and easier to undo.
- Work on a branch; keep `main` working.
- Read `git status` before every `add` and `commit`.
- Never commit secrets: `.env` files, passwords, API keys.
- Never force-push a shared branch.
- About to try something risky? Make a branch first. It costs nothing.

CHAPTER 12

Cheat sheet

Every command in this book, grouped by when you need it.

Setup

Command	What it does
`git config --global user.name "Name"`	Set your name for commits
`git config --global user.email "you@x.com"`	Set your email for commits
`gh auth login`	Log in to GitHub from the terminal

Start

Command	What it does
`git clone <url>`	Download a repo from GitHub
`git init`	Turn the current folder into a repo
`git remote add origin <url>`	Connect a local repo to GitHub

Daily

Command	What it does
`git pull`	Get the latest from GitHub
`git status`	See what's changed and what to do next
`git add <file>` / `git add .`	Stage one file / everything
`git commit -m "msg"`	Save a snapshot
`git push`	Upload commits to GitHub
`git diff`	Show unstaged line changes

Branches and pull requests

Command	What it does
`git branch`	List branches
`git switch -c <name>`	Create a branch and move to it
`git switch <name>`	Move to a branch
`git push -u origin <name>`	Put a new branch on GitHub
`git merge <name>`	Merge that branch into the current one
`git branch -d <name>`	Delete a merged branch

Command	What it does
`git fetch`	Update info from GitHub, files untouched
`gh pr create`	Open a pull request from the terminal
`git merge --abort`	Cancel a merge and go back

Undo

Command	What it does
`git restore <file>`	Throw away unstaged edits
`git restore --staged <file>`	Unstage, keep edits
`git commit --amend`	Change the last commit (local only)
`git reset --soft HEAD~1`	Undo last commit, keep changes (local only)
`git revert <hash>`	Reverse a pushed commit safely
`git stash` / `git stash pop`	Park work / bring it back
`git reflog`	Find "lost" commits

History

Command	What it does
`git log --oneline --graph --all`	Visual commit history
`git show <hash>`	What one commit changed
`git blame <file>`	Who changed each line

CHAPTER 13

Glossary

Plain-English meanings for the words you'll hit in error messages and tutorials, A to Z.

Term	Meaning
Branch	A named line of commits you can work on separately
Clone	Download a full copy of a repo, history included
Commit	A saved snapshot with a message and a unique hash
Detached HEAD	You're looking at an old commit, not on any branch. `git switch main` to get back.
Fast-forward	A merge where Git just moves the branch label ahead because nothing diverged
Fetch	Download new info from the remote without changing your files
Fork	Your own GitHub copy of someone else's repo
Hash (SHA)	A commit's ID, like `a1b2c3d` ; the short form is enough for most commands
HEAD	Where you are right now: usually the latest commit on your current branch
Main	The default branch. Older repos call it `master` .
Merge	Combine one branch's changes into another
Origin	The default name for the remote you cloned from
Pull	Fetch plus merge: get the remote's changes into your branch
Pull request (PR)	A GitHub page proposing to merge one branch into another, with review
Push	Upload your commits to the remote
Rebase	Replay your commits on top of another branch for a straight history; rewrites history
Remote	A copy of the repo somewhere else, usually GitHub
Repository (repo)	A project folder plus its full Git history
Staging area (index)	The list of changes queued for your next commit
Stash	A shelf for unfinished changes you want out of the way
Tag	A permanent label on one commit, usually a version like `v1.0`
Upstream	The branch or repo yours tracks; with forks, the original repo
Working directory	The actual files on your computer

CHAPTER 14

Practice session (do it together)

This drill covers every chapter before it in about an hour, with two people and one shared repo. Person A owns the repo; Person B is the learner. Tick each step off as you go.

Round 1: the basics

- [ ] A creates a repo called `playground` on GitHub with a README (chapter 3, option A).
- [ ] A adds B as a collaborator: **Settings**, **Collaborators**, **Add people**.
- [ ] B accepts the invite (email, or github.com/notifications) and runs `git clone`.
- [ ] B adds a line to the README, then runs `status`, `add`, `commit`, `push` (chapter 4).
- [ ] A runs `git pull` and sees B's line appear.

Round 2: branches and a pull request

- [ ] B runs `git switch -c add-favorites` and creates `favorites.md` listing three favorite things.
- [ ] B commits, pushes the branch, and opens a PR (chapter 6).
- [ ] A reviews it in **Files changed**, leaves a comment, and requests one change.
- [ ] B makes the change, commits, pushes. The PR updates.
- [ ] A approves and merges. Both run `git switch main` and `git pull`.

Round 3: a deliberate merge conflict

- [ ] Both make a branch and change the README's first line to different words.
- [ ] Both push and open PRs. A merges theirs first.
- [ ] B's PR now shows a conflict. B runs `git switch main`, `git pull`, `git switch <branch>`, `git merge main`.
- [ ] B resolves the markers, then `add`, `commit`, `push` (chapter 7). A merges it.

Round 4: undo

- [ ] B commits and pushes a deliberately bad change.
- [ ] B finds its hash with `git log --oneline` and runs `git revert <hash>`, then pushes (chapter 8).

Bonus: put it online

- [ ] Add an `index.html` with some text.
- [ ] In the repo's **Settings**, open **Pages**, choose to deploy from the `main` branch, and save. The site goes live at `username.github.io/playground` after a minute or two.